

Hands-on Assignment 4

2466025 박혜린

Step1. Implement two types of findMaxMin algorithm

```
A = [24, 75, 92, 83, 61, 48, 97, 50]
```

```
# Type 1: 2(n-1) comparisons
# Each element is compared twice: one for min, one for max
def find_max_min_type1(arr):
    comparisons = 0
    mn = mx = arr[0] # initialize with first element (no comparison needed)

    for i in range(1, len(arr)): # iterate n-1 times
        comparisons += 1
        if arr[i] < mn: # comparison 1: min check
            mn = arr[i]

        comparisons += 1
        if arr[i] > mx: # comparison 2: max check
            mx = arr[i]

    return mn, mx, comparisons

# Type 2: floor(3n/2)-2 comparisons
# Split array in half recursively
# merge by comparing only the two candidates
def find_max_min_type2(arr, left, right):
    # Base case 1: single element is both min and max, 0 comparisons
    if left == right:
        return arr[left], arr[left], 0
```

```

    # Base case 2: two elements, 1 comparison determines min and max directly
    if right - left == 1:
        if arr[left] < arr[right]:
            return arr[left], arr[right], 1 # left=min, right=max
        else:
            return arr[right], arr[left], 1 # right=min, left=max

    # Recursive case: split into left and right, solve each, then merge
    mid = (left + right) // 2
    mn_l, mx_l, cmp_l = find_max_min_type2(arr, left, mid)
    # solve left half
    mn_r, mx_r, cmp_r = find_max_min_type2(arr, mid + 1, right) # solve right half

    # Merge: 1 comparison for min, 1 for max = 2 comparisons at this level
    final_min = mn_l if mn_l < mn_r else mn_r
    final_max = mx_l if mx_l > mx_r else mx_r

    return final_min, final_max, cmp_l + cmp_r + 2 # accumulate total comparisons

# Run
print(f"Array: {A} (n = {len(A)})\n")

mn, mx, cmp = find_max_min_type1(A)
print("Type 1")
print(f"  MIN = {mn}, MAX = {mx}")
print(f"  Comparisons: {cmp} (expected 2n-2 = {2*(len(A)-1)})\n")

mn, mx, cmp = find_max_min_type2(A, 0, len(A) - 1)
print("Type 2")

```

```
print(f"  MIN = {mn}, MAX = {mx}")
print(f"  Comparisons: {cmp}  (expected floor(3n/2)-2 = {3*
len(A)//2 - 2})")
```

```
[Running] python -u "c:\Users\hyeri\OneDrive\Desktop\ha4.py"
Array: [24, 75, 92, 83, 61, 48, 97, 50]  (n = 8)

Type 1
  MIN = 24, MAX = 97
  Comparisons: 14  (expected 2n-2 = 14)

Type 2
  MIN = 24, MAX = 97
  Comparisons: 10  (expected floor(3n/2)-2 = 10)

[Done] exited with code=0 in 0.55 seconds
```

Step2. Analyze the time complexities of your implementations

Type1

Loop runs from $i=1$ to $i=n-1$, which is $n-1$ iterations. Each iteration performs total two comparison because of one for min and the other for max.

Total: $(n - 1) * 2$

In this code, $n = 8$, $7*2 = 14$ comparison

Type2

Counting by node type

Pair(leaf) node

- pair calls: $n/2$
- 1 comparison per pair
- $n/2$ pair calls $\times$ 1 comparison = $n/2$

Internal(merge) node

- internal merge calls: $n/2 - 1$
- 2 comparison per pair (one for min and one for max)
- $(n/2 - 1)$ internal merge calls $\times$ 2 comparisons = $n - 2$

Total = $n/2 + (n - 2) = 3n/2 - 2$

In this code, $n = 8$ elements forming 4 leaf-pairs, the tree above them has $4 \rightarrow 2 \rightarrow 1$ structure, giving $2 + 1 = 3$ internal nodes ($= n/2 - 1$).

Therefore, total is , $(3*8)/2 - 2 = 10$.

Step3. Explain why one type performs more efficiently than the other

Comparison between Type 1 and Type 2

Type 2 reduces the number of comparisons for $n=8$. Pair base determine both min and max with a single comparison. Therefore, Type 2 needs only 6 additional comparisons for merging. As a result, Type 2 uses 10 comparisons instead of 14, saving 4 comparisons. However, Type 1 always uses 2 comparisons per element.

Step	Type 1	Type 2
Determining min/max within pairs	2 per element $\times$ 7 elements = 14	1 per pair $\times$ 4 pairs = 4
Merging results	—	2 per merge $\times$ 3 merges = 6
Total	14	10

Type 1: Each element costs 2 comparisons, always

```
if arr[i] < mn:    # comparison 1 – checks only min
if arr[i] > mx:    # comparison 2 – checks only max
```

Every element after the first gets one comparison for min and one comparison for max independently. For $n-1$ elements, that is always only $2(n-1)$ comparisons.

Type 2: Each pair costs 1 comparison, yielding both min and max

```
if right - left == 1:
    comparisons[0] += 1
    if arr[left] < arr[right]:
        return arr[left], arr[right]    # left=min, right=ma
x
    else:
        return arr[right], arr[left]    # right=min, left=ma
x
```

A single comparison between two elements simultaneously determines which is minimum and which is maximum. Therefore, one comparison does the work of two in Type 1. For $n/2$ pairs, this step costs only $n/2$ comparisons instead of n .

The savings come entirely from this step. The merge stage requires additional comparisons, which is $2(n/2 - 1) = n - 2$, the total comparison of $3n/2 - 2$.